

Deep Networks

Convergence

Introduction to Deep Learning · Class 14

Outline

What Convergence Means

The Best Case: Equal Curvature

Ill-conditioning

Momentum

Learning-rate Schedules

Adaptive Learning Rates

Putting It Together

What Convergence Means

From a safe step to a fast one

Class 13 established that a step with $\eta < 2/L$ lowers the error. It said nothing about how long that takes.

Settled in Class 13

$-\nabla E$ is the steepest direction

$\eta < 2/L$ guarantees a decrease

the mini-batch gradient is unbiased

small batches settle in a noise ball

Open today

how many steps to reach the minimum

which η is best, not merely safe

what its noise costs in the limit

how to shrink that ball

Today: the best possible case first, where the answer is one step; then the property of real problems that destroys it, and four things we do about it.

Notation added today

$\lambda_{\min}, \lambda_{\max}$	extreme eigenvalues of $\mathbf{H}$	μ	momentum parameter
$\kappa = \lambda_{\max}/\lambda_{\min}$	condition number	β_1, β_2	Adam's decay parameters
α_i	coefficient along $\mathbf{u}_i$	$\mathbf{s}, \mathbf{r}$	running first and second moments
ρ	contraction factor per step	ε	stabilizing constant, $\sim 10^{-8}$

- Bishop writes the momentum parameter μ and Prince writes it β ; both appear in the literature

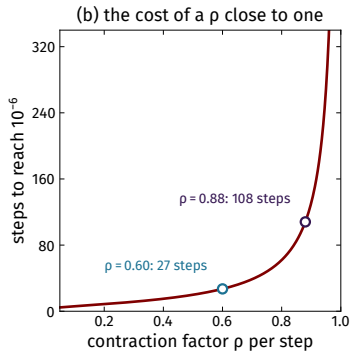
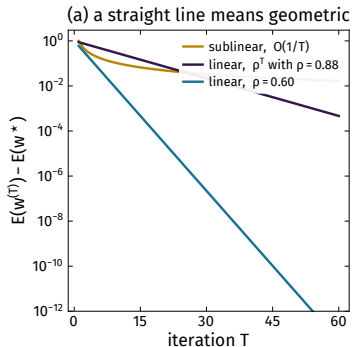
What it means for an iteration to converge

Definition (rate of convergence)

Write $\delta_T = E(\mathbf{w}^{(T)}) - E(\mathbf{w}^*)$. The iteration converges **linearly** (or geometrically) with factor $\rho \in (0, 1)$ if $\delta_T \leq \rho^T \delta_0$, and **sublinearly** if δ_T falls only as a power of T , for instance $\delta_T = O(1/T)$.

- “Linear” is about the *logarithm*: each step multiplies the error by a constant
- The number of steps to reach accuracy ϵ is then $\log(1/\epsilon)/\log(1/\rho)$
- Everything today is a fight to make ρ smaller

Two rates, drawn



(a) error above the optimum against iteration, on a logarithmic axis: a geometric rate is a straight line. (b) how many steps a given contraction factor needs.

The tool: the quadratic model, again

Class 13 used the local quadratic model to say what a minimum looks like. The same model now says how the iterate moves on it.

$$E(\mathbf{w}) = E(\mathbf{w}^*) + \frac{1}{2} \sum_i \lambda_i \alpha_i^2, \quad \mathbf{w} - \mathbf{w}^* = \sum_i \alpha_i \mathbf{u}_i.$$

- $\mathbf{H}\mathbf{u}_i = \lambda_i \mathbf{u}_i$, and the $\mathbf{u}_i$ are orthonormal
- $\alpha_i = \mathbf{u}_i^\top (\mathbf{w} - \mathbf{w}^*)$ is the distance to the minimum along $\mathbf{u}_i$
- The analysis is exact for a quadratic and a good local description near any minimum

Bishop & Bishop (2024), Eqs. 7.7–7.11.

The Best Case: Equal Curvature

The directions never interact

Lemma 1 (Decoupled contraction)

Apply $\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla E$ to the quadratic model. Then each coefficient evolves independently,

$$\alpha_i^{new} = (1 - \eta \lambda_i) \alpha_i^{old}, \quad \text{so} \quad \alpha_i^{(T)} = (1 - \eta \lambda_i)^T \alpha_i^{(0)}.$$

- Differentiating the model gives $\nabla E = \sum_i \lambda_i \alpha_i \mathbf{u}_i$.
- Write the update in the same basis, $\Delta \mathbf{w} = \sum_i \Delta \alpha_i \mathbf{u}_i$.
- Matching coefficients in $\Delta \mathbf{w} = -\eta \nabla E$ and using $\mathbf{u}_i^\top \mathbf{u}_j = \delta_{ij}$ gives $\Delta \alpha_i = -\eta \lambda_i \alpha_i$.
- Iterating that one-line recursion T times gives the stated power. □

Bishop & Bishop (2024), Eqs. 7.24–7.29.

The directions never interact

Lemma 1 (Decoupled contraction)

Apply $\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla E$ to the quadratic model. Then each coefficient evolves independently,

$$\alpha_i^{new} = (1 - \eta \lambda_i) \alpha_i^{old}, \quad \text{so} \quad \alpha_i^{(T)} = (1 - \eta \lambda_i)^T \alpha_i^{(0)}.$$

- Differentiating the model gives $\nabla E = \sum_i \lambda_i \alpha_i \mathbf{u}_i$.
- **Write the update in the same basis, $\Delta \mathbf{w} = \sum_i \Delta \alpha_i \mathbf{u}_i$.**
- Matching coefficients in $\Delta \mathbf{w} = -\eta \nabla E$ and using $\mathbf{u}_i^\top \mathbf{u}_j = \delta_{ij}$ gives $\Delta \alpha_i = -\eta \lambda_i \alpha_i$.
- Iterating that one-line recursion T times gives the stated power. □

Bishop & Bishop (2024), Eqs. 7.24–7.29.

The directions never interact

Lemma 1 (Decoupled contraction)

Apply $\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla E$ to the quadratic model. Then each coefficient evolves independently,

$$\alpha_i^{new} = (1 - \eta \lambda_i) \alpha_i^{old}, \quad \text{so} \quad \alpha_i^{(T)} = (1 - \eta \lambda_i)^T \alpha_i^{(0)}.$$

- Differentiating the model gives $\nabla E = \sum_i \lambda_i \alpha_i \mathbf{u}_i$.
- Write the update in the same basis, $\Delta \mathbf{w} = \sum_i \Delta \alpha_i \mathbf{u}_i$.
- **Matching coefficients in $\Delta \mathbf{w} = -\eta \nabla E$ and using $\mathbf{u}_i^\top \mathbf{u}_j = \delta_{ij}$ gives $\Delta \alpha_i = -\eta \lambda_i \alpha_i$.**
- Iterating that one-line recursion T times gives the stated power. □

Bishop & Bishop (2024), Eqs. 7.24–7.29.

The directions never interact

Lemma 1 (Decoupled contraction)

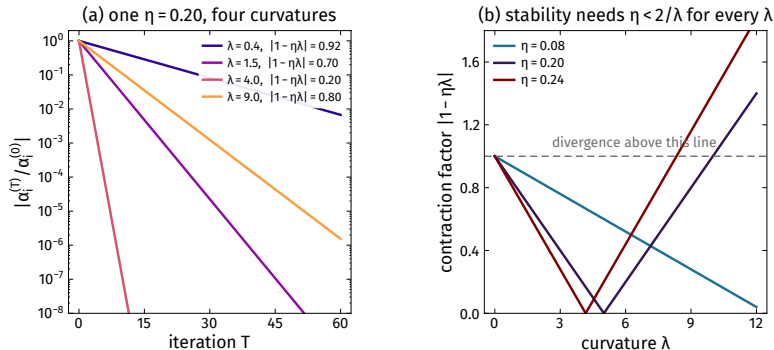
Apply $\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla E$ to the quadratic model. Then each coefficient evolves independently,

$$\alpha_i^{new} = (1 - \eta \lambda_i) \alpha_i^{old}, \quad \text{so} \quad \alpha_i^{(T)} = (1 - \eta \lambda_i)^T \alpha_i^{(0)}.$$

- Differentiating the model gives $\nabla E = \sum_i \lambda_i \alpha_i \mathbf{u}_i$.
- Write the update in the same basis, $\Delta \mathbf{w} = \sum_i \Delta \alpha_i \mathbf{u}_i$.
- Matching coefficients in $\Delta \mathbf{w} = -\eta \nabla E$ and using $\mathbf{u}_i^\top \mathbf{u}_j = \delta_{ij}$ gives $\Delta \alpha_i = -\eta \lambda_i \alpha_i$.
- **Iterating that one-line recursion T times gives the stated power.** □

Bishop & Bishop (2024), Eqs. 7.24–7.29.

One learning rate, many curvatures



(a) the four contraction factors a single η produces on four curvatures. (b) $|1 - \eta\lambda|$ must stay below one for every eigenvalue, so the largest one binds.

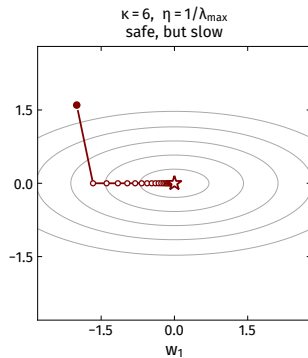
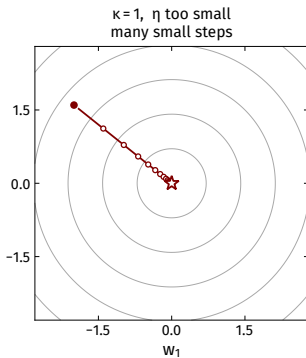
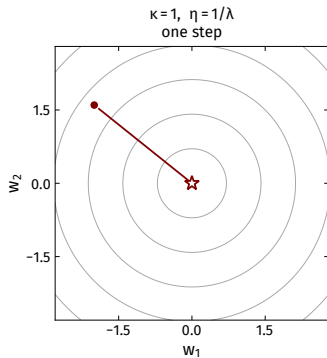
What that recursion already tells us

Corollary 2 (Stability and linear convergence)

The iteration reaches $\mathbf{w}^$ if and only if $|1 - \eta\lambda_i| < 1$ for every i , that is $0 < \eta < 2/\lambda_{\max}$, and it then converges *linearly* with $\rho = \max_i |1 - \eta\lambda_i|$.*

- Class 13 obtained $\eta < 2/L$ from smoothness; on a quadratic $L = \lambda_{\max}$, and the two agree exactly
- $1 - \eta\lambda_i$ may go *negative*: that is oscillation, harmless while the magnitude stays below one
- Convergence also needs every $\lambda_i > 0$, so the stationary point really is a minimum

When every direction curves alike



With $\lambda_1 = \lambda_2$ the contours are circles, the negative gradient points straight at the minimum, and $\eta = 1/\lambda$ arrives in a single step. The last panel shows what a modest spread of curvature already costs.

The best case, stated

- If all λ_i are equal then $\eta = 1/\lambda$ makes every factor $1 - \eta\lambda_i$ exactly zero
- One step. $\rho = 0$. No method in the rest of this class improves on it
- The reason is that $\mathbf{H} = \lambda I$, so $-\nabla E$ points **exactly** at the minimum from everywhere

Everything that follows exists because real error surfaces do not have equal curvature in every direction. The rest of this class measures that failure and repairs it.

Ill-conditioning

The condition number

Definition (condition number of the Hessian)

$$\kappa = \frac{\lambda_{\max}}{\lambda_{\min}} \geq 1.$$

A problem with large κ is called **ill-conditioned**.

- $\kappa = 1$ is the circular bowl of the previous section
- Geometrically, $\sqrt{\kappa}$ is the ratio of the longest to the shortest semi-axis of a contour ellipse
- $\lambda_{\max}$ limits how large η may be; $\lambda_{\min}$ limits how fast the slowest direction moves. One number cannot satisfy both

Bishop & Bishop (2024), §7.3.

The condition number as the shape of a contour

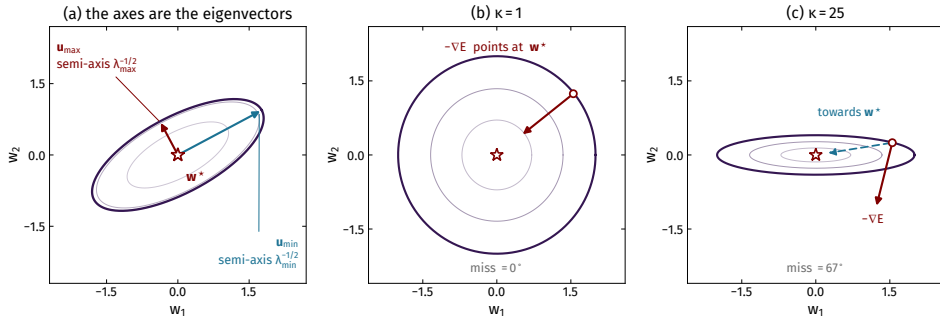
A contour of the quadratic model is the set $\frac{1}{2} \sum_i \lambda_i \alpha_i^2 = c$, where $\alpha_i = \mathbf{u}_i^\top (\mathbf{w} - \mathbf{w}^*)$. That is an **ellipse**: its axes point along the eigenvectors $\mathbf{u}_i$, and the semi-axis along $\mathbf{u}_i$ has length

$$a_i = \sqrt{2c} \lambda_i^{-1/2} \quad \Rightarrow \quad \frac{a_{\max}}{a_{\min}} = \sqrt{\frac{\lambda_{\max}}{\lambda_{\min}}} = \sqrt{\kappa}.$$

- The steeply curving direction gives the *short* axis: large λ , small $\lambda^{-1/2}$
- κ is therefore the elongation of the contour, squared

Bishop & Bishop (2024), Eqs. 7.10–7.14.

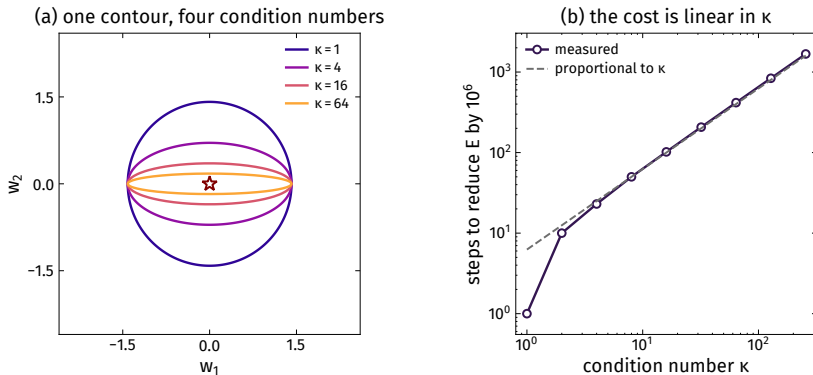
Why the shape matters for descent



(a) the semi-axes of one contour, named. (b) on a circular contour the negative gradient points exactly at w^* .

(c) on an elongated contour it misses by a wide angle, and the step spends most of its length crossing the valley rather than travelling along it.

What ill-conditioning costs



(a) one contour of the same bowl at four condition numbers. (b) steps needed to reduce the error by six orders of magnitude, at the largest safe learning rate, against κ : the count grows in proportion to κ .

The slowest direction sets the pace

Proposition 3 (The rate at the stability limit)

Take η as large as stability allows, $\eta \rightarrow 2/\lambda_{\max}$. The fastest direction then barely contracts at all, and the slowest contracts by

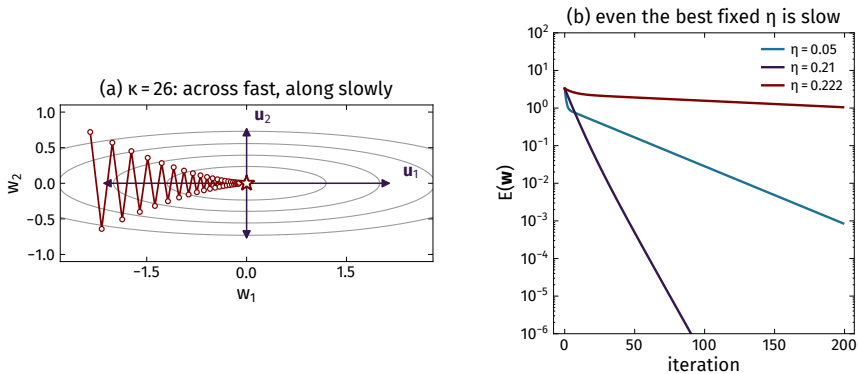
$$\left|1 - \frac{2\lambda_{\min}}{\lambda_{\max}}\right| = \left|1 - \frac{2}{\kappa}\right| \quad \text{per step.}$$

For large κ this is $1 - 2/\kappa$: a factor that approaches one as the problem worsens.

- Pushing η to its limit buys speed in the direction that was never the problem; the long axis of the ellipse is the one that needs the steps

Bishop & Bishop (2024), Eq. 7.30.

The valley



(a) with $\kappa \approx 26$ the negative gradient points almost across the valley rather than along it, so successive steps oscillate. (b) three fixed learning rates on that surface: none of them is fast.

Where the condition number comes from

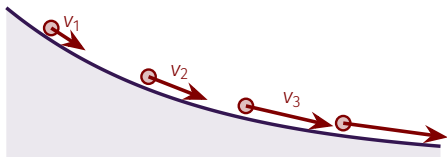
- **Input scale**: a feature measured in millimetres and one measured in kilometres produce curvatures differing by 10^{12}
- **Correlated inputs**: two nearly identical features make one eigenvalue nearly zero
- **Depth**: the same weight change has a different effect early and late in a network, so layers differ in curvature by orders of magnitude
- **Output scale**: an unnormalised target inflates every eigenvalue at once

Two responses are possible. Change the *iteration*, which is the rest of this class. Or change the *problem* so that κ is small to begin with, which is Class 15.

Momentum

A ball with inertia

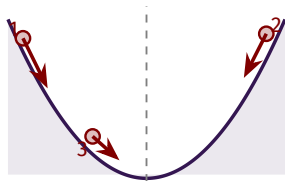
(a) a long, gentle slope



the slope pushes the same way each time,
so the pushes *add*

$v_1 < v_2 < v_3$: the ball builds speed

(b) a narrow gully



the slope reverses at every crossing,
so the pushes *cancel*

the sideways swing dies away

The same rule does both: it accelerates where the slope is consistent and stays out of the way where it alternates.

The analogy, term by term

A ball rolling downhill

it keeps some of its previous velocity

the slope adds to that velocity

a long consistent slope builds speed

a narrow gully cancels sideways motion

In the update

$$\mu \Delta \mathbf{w}^{(\tau-2)}$$

$$-\eta \nabla E$$

aligned gradients accumulate

alternating gradients cancel

- A ball does not stop and re-measure the slope at every instant, and neither should the iterate
- The single parameter μ decides how much of the past to keep

The momentum update

$$\Delta \mathbf{w}^{(\tau-1)} = -\eta \nabla E(\mathbf{w}^{(\tau-1)}) + \mu \Delta \mathbf{w}^{(\tau-2)}, \quad \mathbf{w}^{(\tau)} = \mathbf{w}^{(\tau-1)} + \Delta \mathbf{w}^{(\tau-1)}$$

- $\mu \in [0, 1)$; a typical value is 0.9
- Unrolling the recursion shows $\Delta \mathbf{w}$ is an exponentially weighted sum of **every** past gradient
- Prince writes the same idea as a running average, $m \leftarrow \beta m + (1 - \beta) \nabla E$ and $\mathbf{w} \leftarrow \mathbf{w} - \eta \mathbf{m}$, which rescales η by $(1 - \beta)$ but is otherwise identical

Bishop & Bishop (2024), Eq. 7.31; Prince (2023), Eq. 6.11.

Why momentum helps where it helps

Proposition 4 (Effective learning rate on a consistent slope)

If the gradient is essentially unchanged over a long run of steps, the momentum update converges to

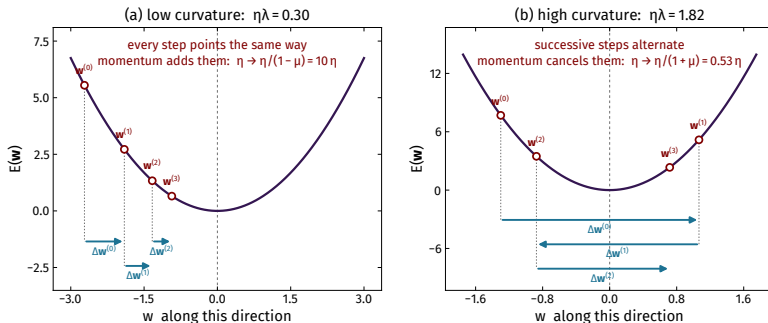
$$\Delta \mathbf{w} = -\eta \nabla E \{1 + \mu + \mu^2 + \dots\} = -\frac{\eta}{1 - \mu} \nabla E,$$

so the effective learning rate is $\eta/(1 - \mu)$. With $\mu = 0.9$ that is a tenfold increase.

- Where successive gradients **alternate** in sign the terms cancel instead, and the effective rate stays near η
- That asymmetry is the whole mechanism: acceleration along the valley, no amplification across it

Bishop & Bishop (2024), Eqs. 7.32–7.33.

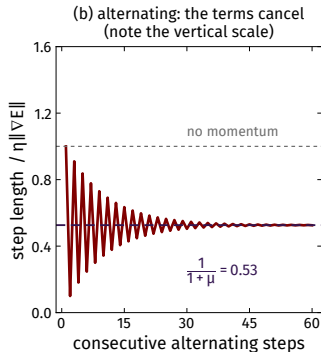
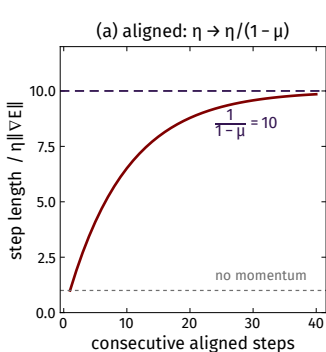
The two regimes



Plain gradient descent at one fixed η , along two directions of the same surface. (a) low curvature: the displacements $\Delta w^{(k)}$ all point the same way. (b) high curvature: the same η overshoots, so they alternate.

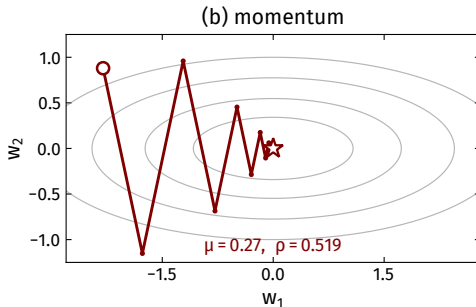
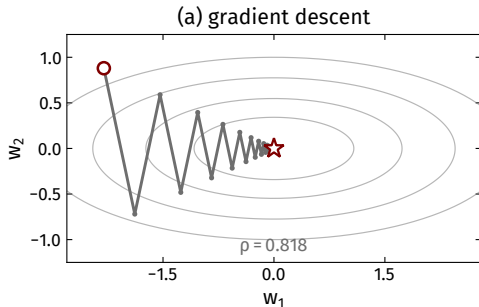
Following the geometry of Bishop & Bishop (2024), Figs. 7.4 and 7.5.

The same two regimes, measured



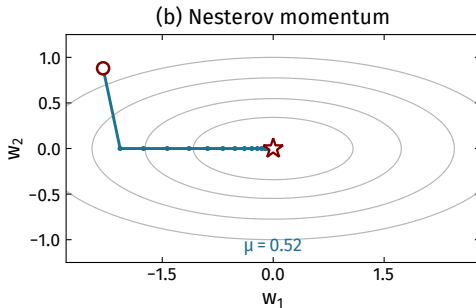
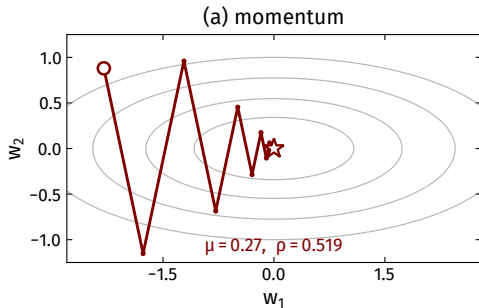
The length of the accumulated step, in units of $\eta \|\nabla E\|$. (a) with the gradient pointing the same way each time it approaches $1/(1 - \mu)$. (b) with the gradient alternating it settles near $1/(1 + \mu)$ — below one, not above.

Momentum on the valley



The same surface and start, each method at the constants that are optimal for it; the first twenty iterates are marked. Momentum crosses the valley fewer times and travels much further along it per step, and the contraction factor falls from 0.818 to 0.519.

Nesterov on the same valley



Momentum repeated on the left as the reference. Nesterov removes the crossing of the valley in a single step and then runs straight along the floor, because the gradient is measured at the look-ahead point rather than at the current one.

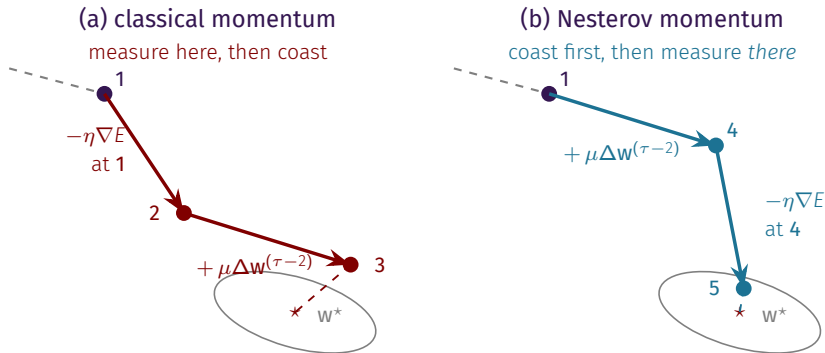
Nesterov momentum

$$\Delta \mathbf{w}^{(\tau-1)} = -\eta \nabla E \left(\mathbf{w}^{(\tau-1)} + \mu \Delta \mathbf{w}^{(\tau-2)} \right) + \mu \Delta \mathbf{w}^{(\tau-2)}$$

- The two ingredients are unchanged: a momentum carry $\mu \Delta \mathbf{w}^{(\tau-2)}$ and a gradient step $-\eta \nabla E$
- Only the **order** changes. Classical momentum measures the slope where the iterate is; Nesterov carries first and measures the slope where it is about to be
- Because the look-ahead point is already past the turn, the gradient measured there points back, and the overshoot is corrected within the same step

Bishop & Bishop (2024), Eq. 7.34; Prince (2023), Eq. 6.12.

Nesterov momentum, one step drawn



Both steps use the same two vectors. Taking the carry first puts the gradient measurement past the turn, so point 5 lands nearer w^* than point 3.

Stochastic gradient descent with momentum

Algorithm 5 (SGD with momentum)

Input: data indexed by n ; batch size B ; per-batch error $E_{n:n+B-1}$; learning rate η ; momentum μ ; initial $\mathbf{w}$

$n \leftarrow 1$; $\Delta \mathbf{w} \leftarrow 0$

repeat

$\Delta \mathbf{w} \leftarrow -\eta \nabla E_{n:n+B-1}(\mathbf{w}) + \mu \Delta \mathbf{w}$ *// update term*

$\mathbf{w} \leftarrow \mathbf{w} + \Delta \mathbf{w}$ *// weight vector update*

$n \leftarrow n + B$

if $n > N$ *then* shuffle data; $n \leftarrow 1$ *end if*

until convergence; *return* $\mathbf{w}$

Bishop & Bishop (2024), Algorithm 7.3.

Learning-rate Schedules

Why a fixed step size cannot finish the job

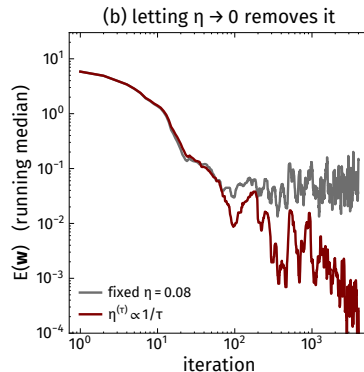
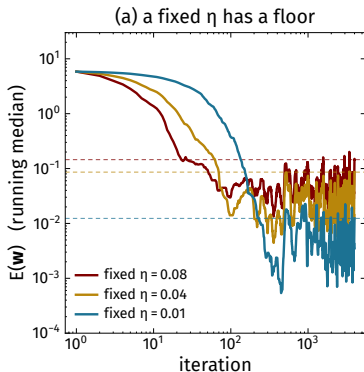
Momentum attacks the condition number. It does nothing about the noise in a mini-batch gradient.

- The mini-batch gradient's variance does not shrink as $\mathbf{w}$ approaches $\mathbf{w}^*$, so near the minimum the signal vanishes while the noise does not
- The iterate then wanders in a ball whose radius is set by η ; halving η halves the radius but never removes it
- Large η explores, small η refines — we want both, in that order

So make η a function of the step index: $\mathbf{w}^{(\tau)} = \mathbf{w}^{(\tau-1)} - \eta^{(\tau-1)} \nabla E_n(\mathbf{w}^{(\tau-1)})$.

Bishop & Bishop (2024), Eq. 7.35; Prince (2023), §6.2.2.

The noise floor



(a) three fixed learning rates on the same noisy problem: each stops improving at a level proportional to η .

(b) the same problem with η decayed as $1/\tau$, which keeps descending past every one of those floors.

Three algebraic schedules

linear

$$\eta^{(\tau)} = \left(1 - \frac{\tau}{K}\right)\eta^{(0)} + \frac{\tau}{K}\eta^{(K)}$$

power law

$$\eta^{(\tau)} = \eta^{(0)} \left(1 + \frac{\tau}{s}\right)^{-c}$$

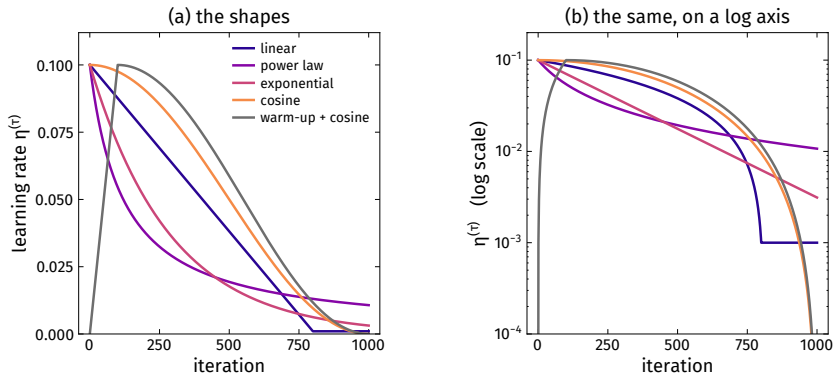
exponential

$$\eta^{(\tau)} = \eta^{(0)} c^{\tau/s}$$

- The linear form decays over K steps and is then held at $\eta^{(K)}$
- $\eta^{(0)}$, $\eta^{(K)}$, K , s and c are found empirically
- Watching the learning curve is the practical test: it should be falling at a steady rate, not flattening early

Bishop & Bishop (2024), Eqs. 7.36–7.38.

The schedules in use



The three algebraic forms together with the two shapes most used in practice: cosine annealing, and a linear warm-up followed by cosine annealing.

Warm-up, and why the start is different

- Cosine annealing decays smoothly to zero over the whole run and has no schedule parameters to tune beyond its length
- **Warm-up** raises η from near zero over the first few thousand steps, then anneals
- The reason is that adaptive methods estimate gradient statistics from very few samples at the start, so those estimates are unreliable exactly when the steps are largest
- Warm-up is also standard when training with very large batches

Loshchilov & Hutter (2017); Goyal et al. (2018).

Adaptive Learning Rates

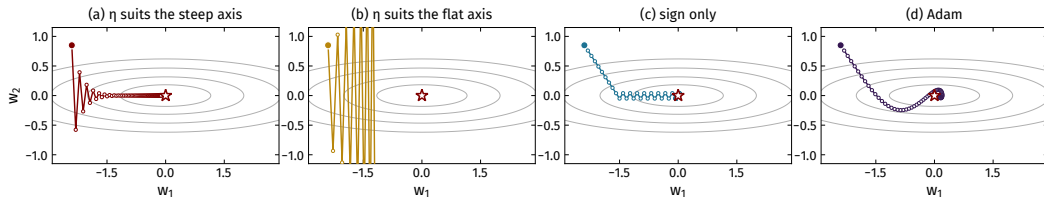
One rate cannot serve two curvatures

Momentum and schedules both keep a single η shared by every parameter. That is the next assumption to drop.

- Gradient descent makes **large** adjustments where the gradient is large — where it should perhaps be cautious
- And **small** adjustments where the gradient is small — where it should perhaps explore
- If the surface is far steeper in one direction than another, no single η makes good progress in both and stays stable
- The fix: give every parameter its own learning rate, adjusted automatically from what its gradients have been doing

Prince (2023), §6.4; Bishop & Bishop (2024), §7.3.3.

The problem, and three responses



(a) an η chosen for the steep direction leaves the flat one barely moving. (b) an η chosen for the flat direction is unstable. (c) using only the sign of each component moves a fixed distance along each axis, and makes progress in both, but cannot settle. (d) Adam smooths both quantities and does settle.

Normalising the gradient to its sign

$$\mathbf{m}^{(\tau+1)} = \nabla E, \quad \mathbf{v}^{(\tau+1)} = (\nabla E)^2, \quad \mathbf{w}^{(\tau+1)} = \mathbf{w}^{(\tau)} - \eta \frac{\mathbf{m}^{(\tau+1)}}{\sqrt{\mathbf{v}^{(\tau+1)}} + \varepsilon}$$

- The root and the division are **element-wise**, so every component reduces to its sign, and the step is a fixed distance η along each axis
- $\varepsilon \sim 10^{-8}$ prevents division by zero
- It progresses in every direction and **cannot converge**: it bounces around the minimum forever

Prince (2023), Eqs. 6.13–6.14.

AdaGrad

$$r_i^{(\tau)} = r_i^{(\tau-1)} + \left(\frac{\partial E}{\partial w_i} \right)^2, \quad \mathbf{w}_i^{(\tau)} = \mathbf{w}_i^{(\tau-1)} - \frac{\eta}{\sqrt{r_i} + \varepsilon} \left(\frac{\partial E}{\partial w_i} \right)$$

- Each parameter accumulates the sum of its own squared gradients, from $r_i^{(0)} = 0$
- Parameters that have seen large gradients — high curvature — have their rates reduced most
- **The flaw:** r_i accumulates from the first step and never forgets, so the steps eventually become too small and learning stalls

Duchi, Hazan & Singer (2011); Bishop & Bishop (2024), Eqs. 7.39–7.40.

RMSProp

$$r_i^{(\tau)} = \beta r_i^{(\tau-1)} + (1 - \beta) \left(\frac{\partial E}{\partial w_i} \right)^2, \quad \mathbf{w}_i^{(\tau)} = \mathbf{w}_i^{(\tau-1)} - \frac{\eta}{\sqrt{r_i} + \varepsilon} \left(\frac{\partial E}{\partial w_i} \right)$$

- One change from AdaGrad: the running *sum* becomes a running *average*, with $0 < \beta < 1$ and typically $\beta = 0.9$
- The estimate now forgets on a timescale of about $1/(1 - \beta)$ steps, so it tracks the curvature the iterate is currently in
- The learning rate can go back up when the surface flattens, which is exactly what AdaGrad could not do

Hinton (2012); Bishop & Bishop (2024), Eqs. 7.41–7.42.

Adam

$$\begin{aligned}s_i^{(\tau)} &= \beta_1 s_i^{(\tau-1)} + (1 - \beta_1) \frac{\partial E}{\partial w_i} & r_i^{(\tau)} &= \beta_2 r_i^{(\tau-1)} + (1 - \beta_2) \left(\frac{\partial E}{\partial w_i} \right)^2 \\ \hat{s}_i &= \frac{s_i^{(\tau)}}{1 - \beta_1^\tau} & \hat{r}_i &= \frac{r_i^{(\tau)}}{1 - \beta_2^\tau} & \mathbf{w}_i^{(\tau)} &= \mathbf{w}_i^{(\tau-1)} - \frac{\eta \hat{s}_i}{\sqrt{\hat{r}_i} + \epsilon}\end{aligned}$$

- RMSProp with momentum: an exponential average of the gradient *and* of its square. The name is short for **adaptive moments**
- Typical values are $\beta_1 = 0.9$ and $\beta_2 = 0.99$
- The most widely used training algorithm in deep learning

Kingma & Ba (2015); Bishop & Bishop (2024), Eqs. 7.43–7.47.

Why the bias correction is there

Proposition 6 (The averages start too small)

With $\mathbf{s}^{(0)} = 0$ and a gradient $\mathbf{g}$ that is constant in expectation,

$$\mathbb{E}[\mathbf{s}^{(\tau)}] = (1 - \beta_1^\tau) \mathbf{g},$$

so $\mathbf{s}^{(\tau)}$ underestimates $\mathbf{g}$ by exactly the factor $1 - \beta_1^\tau$. Dividing by that factor removes the bias, and since $\beta_1 < 1$ the correction tends to one as τ grows.

- **Unrolling gives** $\mathbf{s}^{(\tau)} = (1 - \beta_1) \sum_{k=0}^{\tau-1} \beta_1^k \mathbf{g}^{(\tau-k)}$.
- Taking expectations and summing the geometric series gives $(1 - \beta_1) \frac{1 - \beta_1^\tau}{1 - \beta_1} \mathbf{g}$.
- Dividing by $1 - \beta_1^\tau$ therefore returns $\mathbf{g}$ exactly; the same argument applies to r with β_2 . □

Bishop & Bishop (2024), Eqs. 7.45–7.46; Prince (2023), Eq. 6.16.

Why the bias correction is there

Proposition 6 (The averages start too small)

With $\mathbf{s}^{(0)} = 0$ and a gradient $\mathbf{g}$ that is constant in expectation,

$$\mathbb{E}[\mathbf{s}^{(\tau)}] = (1 - \beta_1^\tau) \mathbf{g},$$

so $\mathbf{s}^{(\tau)}$ underestimates $\mathbf{g}$ by exactly the factor $1 - \beta_1^\tau$. Dividing by that factor removes the bias, and since $\beta_1 < 1$ the correction tends to one as τ grows.

- Unrolling gives $\mathbf{s}^{(\tau)} = (1 - \beta_1) \sum_{k=0}^{\tau-1} \beta_1^k \mathbf{g}^{(\tau-k)}$.
- **Taking expectations and summing the geometric series gives $(1 - \beta_1) \frac{1 - \beta_1^\tau}{1 - \beta_1} \mathbf{g}$.**
- Dividing by $1 - \beta_1^\tau$ therefore returns $\mathbf{g}$ exactly; the same argument applies to r with β_2 . □

Bishop & Bishop (2024), Eqs. 7.45–7.46; Prince (2023), Eq. 6.16.

Why the bias correction is there

Proposition 6 (The averages start too small)

With $\mathbf{s}^{(0)} = 0$ and a gradient $\mathbf{g}$ that is constant in expectation,

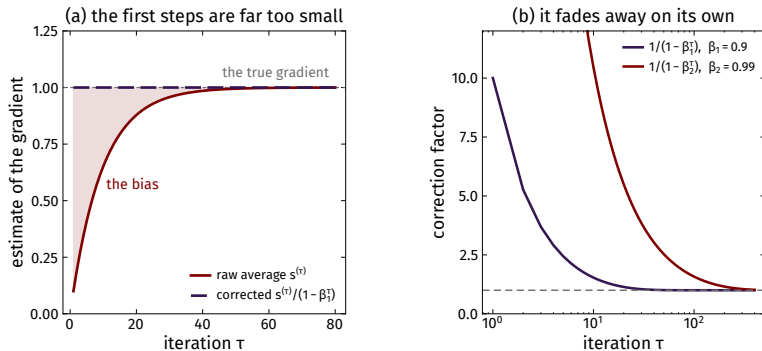
$$\mathbb{E}[\mathbf{s}^{(\tau)}] = (1 - \beta_1^\tau) \mathbf{g},$$

so $\mathbf{s}^{(\tau)}$ underestimates $\mathbf{g}$ by exactly the factor $1 - \beta_1^\tau$. Dividing by that factor removes the bias, and since $\beta_1 < 1$ the correction tends to one as τ grows.

- Unrolling gives $\mathbf{s}^{(\tau)} = (1 - \beta_1) \sum_{k=0}^{\tau-1} \beta_1^k \mathbf{g}^{(\tau-k)}$.
- Taking expectations and summing the geometric series gives $(1 - \beta_1) \frac{1 - \beta_1^\tau}{1 - \beta_1} \mathbf{g}$.
- **Dividing by $1 - \beta_1^\tau$ therefore returns $\mathbf{g}$ exactly; the same argument applies to r with β_2 .** □

Bishop & Bishop (2024), Eqs. 7.45–7.46; Prince (2023), Eq. 6.16.

The correction, drawn



(a) the running average of a constant gradient, with and without the correction. (b) the size of the correction factor against the iteration, for both decay parameters.

Adam, as an algorithm

Algorithm 7 (Adam optimization)

Input: batch size B ; per-batch error $E_{n:n+B-1}$; learning rate η ; decay parameters β_1, β_2 ; stabilizer ϵ

$\mathbf{s} \leftarrow \mathbf{0}$; $\mathbf{r} \leftarrow \mathbf{0}$; $\tau \leftarrow 0$

repeat

choose a mini-batch at random; $\mathbf{g} \leftarrow \nabla E_{n:n+B-1}(\mathbf{w})$; $\tau \leftarrow \tau + 1$

$\mathbf{s} \leftarrow \beta_1 \mathbf{s} + (1 - \beta_1) \mathbf{g}$; $\mathbf{r} \leftarrow \beta_2 \mathbf{r} + (1 - \beta_2) \mathbf{g} \odot \mathbf{g}$ // element-wise

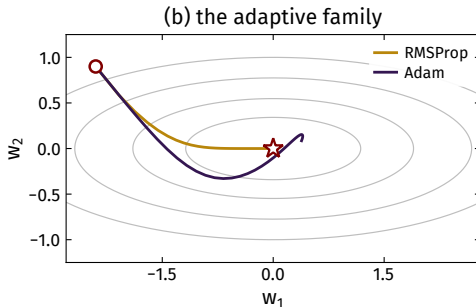
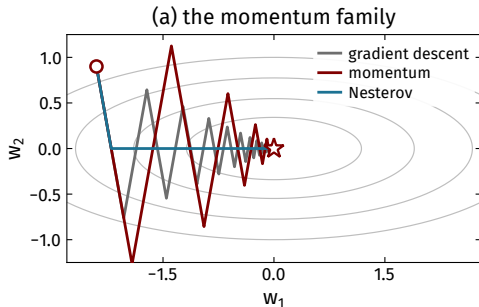
$\hat{\mathbf{s}} \leftarrow \mathbf{s} / (1 - \beta_1^\tau)$; $\hat{\mathbf{r}} \leftarrow \mathbf{r} / (1 - \beta_2^\tau)$ // bias correction

$\mathbf{w} \leftarrow \mathbf{w} - \eta \hat{\mathbf{s}} / (\sqrt{\hat{\mathbf{r}}} + \epsilon)$ // element-wise

until convergence; **return** $\mathbf{w}$

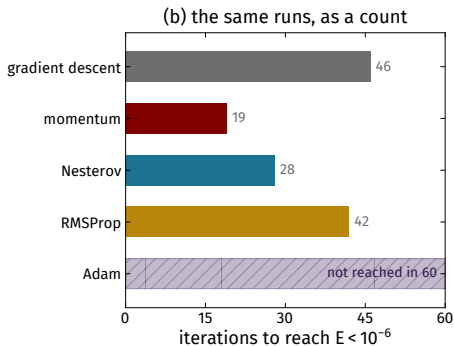
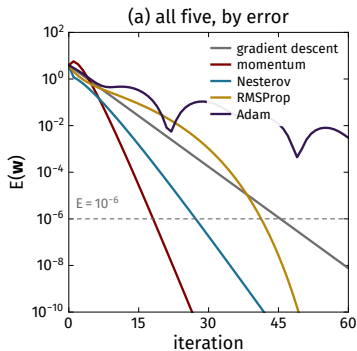
Bishop & Bishop (2024), Algorithm 7.4.

Five update rules, by family



The same ill-conditioned quadratic and start throughout, each method at constants that suit it. (a) the momentum family attacks the ratio $\lambda_{\max}/\lambda_{\min}$, and still crosses the valley. (b) the adaptive family rescales each coordinate instead, and turns along the floor rather than across it.

What the five cost



(a) the same five runs by error. (b) iterations needed to reach $E < 10^{-6}$; Adam does not get there inside the budget. Why the families differ matters more than the ordering on any one surface.

Four honest caveats

- The per-parameter argument assumes the principal curvature directions are **aligned with the axes** of weight space — a diagonal Hessian, which is unlikely. The methods work anyway
- Adam's original convergence proof was for convex losses; Reddi et al. (2018) exhibited a counterexample and proposed AMSGrad, which does converge
- Adam interacts badly with L^2 regularization; AdamW separates the two and is the common default
- Evidence runs in both directions on whether SGD with momentum generalizes better than Adam; much of the gap disappears once Adam's hyperparameters are tuned

Putting It Together

What each method actually fixes

Method	Attacks	How
gradient descent	—	one rate for every direction
momentum	the condition number	inertia along a consistent slope
Nesterov momentum	the condition number	the gradient at the predicted point
schedules	the gradient noise	η decays with the step index
RMSProp, Adam	unequal curvature per parameter	a rate per parameter
normalization (Class 15)	κ itself	changes the problem

The first four change the *iteration* and leave the surface alone. Only the last changes the surface — which is why it is the subject of the next class.

The hyperparameters of the training algorithm

Choice	Typical	What it trades
optimizer	Adam, or SGD with momentum	speed against final quality
learning rate η	10^{-3} (Adam), 10^{-1} (SGD)	stability against progress
schedule	cosine, with warm-up	exploration against refinement
momentum μ	0.9	smoothing against responsiveness
β_1, β_2	0.9, 0.99	as above, per parameter
batch size B	a power of two	noise against cost (Class 13)

- These belong to the **algorithm**, not to the model, and are usually chosen by training several models and comparing on held-out data

Prince (2023), §6.5.

Takeaways

1. On a quadratic the eigen-directions contract independently by $(1 - \eta\lambda_i)$, so one η must serve every curvature at once (Lemma 1)
2. Stability is set by $\lambda_{\max}$ and speed by $\lambda_{\min}$; with equal curvature one step suffices
3. The condition number κ measures the gap, and it is what makes progress along the long axis slow
4. Momentum accelerates along a consistent slope by $1/(1 - \mu)$ and cancels across an oscillating one (Proposition 4)
5. A fixed η leaves a noise floor, and a decaying schedule removes it
6. Adaptive methods give each parameter its own rate from its own gradient history; Adam is RMSProp with momentum and a bias correction

Next class: rather than repairing the iteration, we rescale the network so that κ is small to begin with.

References

- C. M. Bishop & H. Bishop, *Deep Learning: Foundations and Concepts*, Springer, 2024. §7.3 (Eqs. 7.24–7.47, Algorithms 7.3–7.4, Figs. 7.3–7.6).
- S. J. D. Prince, *Understanding Deep Learning*, MIT Press, 2023. §6.3–6.5 (Eqs. 6.11–6.18, Figs. 6.7–6.9).
- B. T. Polyak, “Some methods of speeding up the convergence of iteration methods”, *USSR Comp. Math.*, 1964.
- Y. Nesterov, “A method for solving the convex programming problem with convergence rate $O(1/k^2)$ ”, 1983.
- H. Robbins & S. Monro, “A stochastic approximation method”, *Ann. Math. Statist.*, 1951.
- J. Duchi, E. Hazan & Y. Singer, “Adaptive subgradient methods”, *JMLR*, 2011.
- D. P. Kingma & J. Ba, “Adam: a method for stochastic optimization”, *ICLR*, 2015.
- S. J. Reddi, S. Kale & S. Kumar, “On the convergence of Adam and beyond”, *ICLR*, 2018.
- I. Loshchilov & F. Hutter, “SGDR”, *ICLR*, 2017; “Decoupled weight decay regularization”, *ICLR*, 2019.

Questions?